Comic Craft

Generative AI Comic Strip Maker

PROJECT DEVELOPMENT PHASES – NAAN MUDHALVAN

Project Name

Comic Craft – A Generative AI Powered Comic Strip Creator

Purpose of the Project

The purpose of Comic Craft is to demonstrate how generative AI can be combined with web technologies to automatically transform a simple text idea into a complete visual comic strip. Instead of manually writing dialogue, choosing character expressions, and drawing panels, the user simply types a one-line story idea, and the system uses an AI language model to generate a structured comic script (characters, moods, dialogue, and captions), which is then rendered live as SVG comic art in the browser. The project showcases practical use of generative AI for creative content generation, not just text.

Use / Benefits of the Project

Helps students, educators, and storytellers create comic strips instantly without any drawing or design skill. Demonstrates real-world application of generative AI beyond chatbots – for creative and visual content generation. Useful as a rapid ideation and storyboarding tool for advertisements, awareness campaigns, and classroom teaching aids. It is a fully self-contained single HTML file – no installation, server, or database required, making it easy to run and share. It also includes an offline fallback story generator, so the tool keeps working even if the AI service is unavailable, while encouraging creativity and reducing the technical barrier to producing visual storytelling content.

Project Execution / Working Process

1. The user enters a story idea, selects a comic style (Adventure, Comedy, Noir, Sci-Fi), and chooses the number of panels (4 or 6).
2. On clicking “Generate Comic”, the idea and style are sent as a prompt to an AI language model, which returns a structured JSON script containing, for each panel, a character name, an emotional mood, a short dialogue line, a caption, and a setting.
3. JavaScript functions in the page interpret this JSON and procedurally draw each panel as an SVG graphic – generating a halftone-dot background, a cartoon character face whose expression changes based on the mood field, a speech bubble containing the dialogue, and a caption strip at the bottom of the panel.
4. All generated panels are arranged into a responsive comic-strip grid and displayed instantly on the same page.
5. If the AI service cannot be reached, a built-in rule-based fallback script generator produces a coherent mini-story instead, so the application never fails to produce an output.

Final Output

A single working web application (comic-craft-ai.html) that takes a plain-text story idea as input and outputs a complete, styled comic strip – with AI-generated dialogue, character expressions, and captions rendered as scalable SVG panels – viewable instantly in any browser.

Project Development Phases

1. Brainstorming & Ideation Phase

The project began with the theme “Comic Craft – Generative AI.” The team brainstormed ways generative AI could help ordinary users create visual, creative content rather than just text answers. Several ideas were discussed, including an AI image generator, an AI story writer, and an AI-assisted comic creator. The comic-strip concept was chosen because it uniquely combines two forms of generative AI output in one flow: AI-written dialogue/story content, and a structured visual layout (characters, moods, and panels) built from that content. It was also decided early on that the tool should be a single, portable web page that runs instantly in any browser, with no installation, so it would be easy to build, test, and demonstrate.

2. Requirement Analysis Phase

Once the idea was finalized, the requirements were listed out clearly. On the functional side, the application needed to accept a story idea, a comic style, and a panel count from the user; send this information to a generative AI model to produce a structured comic script; and then visually render that script as comic panels with characters, expressions, dialogue, and captions. On the non-functional side, the tool needed to be fully responsive for mobile and desktop, lightweight enough to load instantly, and resilient – meaning it should keep working even if the AI service is temporarily unavailable. This analysis led to the choice of HTML5, CSS3, JavaScript, inline SVG graphics, and an AI text-generation API as the core technologies for the build.

3. Project Design Phase

In the design phase, the visual identity and information structure of the app were planned before any code was written. A comic-book aesthetic was chosen: a warm cream paper background, bold black ink outlines, a hot-pink accent color, and a bold display font (Bangers) for headings to give the page a hand-drawn comic feel. The page layout was planned as a hero header, an input form (idea, style, panel count), a generate button, and a responsive grid of comic panels below it. The data model for each panel was also designed at this stage – every panel would carry a character name, an emotional mood, a short dialogue line, a caption, and a setting keyword, which together drive how that panel is drawn.

4. Project Planning Phase

The build was planned as four connected modules: the user interface and form, the AI script-generation function, the SVG panel-rendering engine, and a fallback story generator for offline use. The overall data flow was mapped out as: capture user input → send prompt to the AI model → parse the returned JSON script → render each panel as SVG → display the finished comic strip. It was also planned that any failure in the AI call should automatically and silently fall back to a rule-based script generator, so the page never breaks for the user. To keep the first version manageable, the

scope was limited to 4 or 6 panels and four comic styles (Adventure, Comedy, Noir, Sci-Fi).

5. Project Development Phase

Development started with the HTML structure and CSS styling for the form, buttons, and the responsive comic panel grid. The generateScript() function was then built to send a carefully worded prompt to the AI model, requesting strict JSON output containing the panel-by-panel story. Next, the SVG rendering engine was implemented: characterSVG() draws a cartoon face whose eyebrows and mouth shape change based on the mood field, speechBubble() draws a dialogue bubble with a directional tail, and halftonePattern() draws the comic-style dotted panel background. A wrapText() helper was added to automatically wrap long dialogue and captions across multiple lines, and a fallbackScript() function was written to generate a coherent mini-story locally whenever the AI is unreachable. Finally, the Generate button was wired to run this entire pipeline and display the completed comic strip with its title.

6. Project Testing Phase

The application was tested with a variety of story ideas, comic styles, and panel counts to confirm that the AI consistently returned valid, well-formed JSON that the renderer could use. The offline fallback path was tested separately by simulating an unavailable AI service, confirming that the page still produced a complete, readable comic strip. The layout was tested across mobile and desktop screen widths, along with light and dark color schemes, to verify the design stayed readable and properly aligned. Edge cases were also checked, including an empty idea field, unusually long dialogue text, and repeated rapid clicks on the Generate button, to make sure the interface degraded gracefully in every case.

7. Project Documentation Phase

This phase involved writing clear documentation of the project’s purpose, benefits, working process, and final output, along with this explanation of every development phase. The JavaScript source code was commented at each key function – script generation, panel rendering, and text wrapping – to make the logic easy to follow for anyone reviewing or extending the project later. The finished HTML file was also organized as a single, self-contained deliverable, making it simple to package and submit for evaluation.

8. Project Demonstration Phase

For demonstration, comic-craft-ai.html is opened directly in a browser to show the live interface. A sample story idea is entered, a comic style and panel count are selected, and the Generate Comic button is clicked to show the AI scripting and drawing a complete comic strip in real time. The demonstration highlights how different ideas and styles change the characters, dialogue, moods, and captions produced, and also explains the built-in offline fallback, showing that the tool remains fully functional even without access to the AI service.